

Análisis de Datos Empresariales

Trabajo práctico N°2

Grupo 6

Integrantes:

- Acosta, Gonzalo Exequiel.
- López Gutierrez, Daniel Benjamín.
- Machado Benítez, Matías Emmanuel.
- Martin Rodich, Celeste.
- Sosa, Maria Fernanda.

Data Warehouse: Universidad

1. Estrategia de carga inicial

Patrón elegido: Staging Buffer → Full Truncate → Dimensional Load.

La estrategia implementada corresponde a una carga inicial completa o *full load*. Esto implica que, en cada ejecución del proceso ETL, las tablas del Data Warehouse son vaciadas y reconstruidas desde cero a partir de la información presente en staging.

El proceso comienza eliminando el contenido previo del Data Warehouse mediante sentencias `TRUNCATE TABLE`. Debido a la existencia de relaciones de integridad referencial entre dimensiones y hechos, temporalmente se deshabilitan las validaciones de claves foráneas. Posteriormente se ejecuta la limpieza y recarga total de las dimensiones y tablas de hechos.

Esta estrategia fue seleccionada debido a que el proyecto corresponde a una primera carga histórica del entorno analítico. Asimismo, esta estrategia simplifica el control de consistencia y evita problemas derivados de sincronización parcial entre dimensiones y hechos durante las primeras etapas del proyecto.

Se siguen los siguientes puntos claves:

- **Separación de responsabilidades:** la capa Staging absorbe los datos crudos tal como vienen (tipo `VARCHAR`), mientras la capa de transformación es responsable del tipado, validación y modelado dimensional.
- **Datos de origen en crudo:** todas las columnas de Staging llevan el sufijo `_raw`, indicando explícitamente que no han sido transformadas. Se agregan metadatos automáticos: `archivo_origen` y `fecha_carga`.
- **Truncate ordenado por dependencias FK:** antes de la carga, se vacían primero las tablas de hechos y luego las dimensiones, para evitar errores de integridad referencial.
- **Surrogate Keys auto-generadas:** el DWH no reutiliza las claves naturales de Staging. Genera sus propias claves surrogate para aislar el modelo dimensional de los IDs operacionales.

2. Estrategia de carga incremental

El diseño de la carga incremental permite la sincronización periódica entre el Data Warehouse y las fuentes de información sin recurrir a una regeneración total del repositorio. A diferencia de la operación *TRUNCATE-and-load* de la carga inicial, esta estrategia se fundamenta en un enfoque *delta-based ETL*, el cual consiste en identificar y procesar las novedades (registros insertados, actualizados o removidos), aplicando una lógica de consistencia para preservar la fidelidad histórica. Esta estrategia conlleva las siguientes definiciones:

- El DWH funciona de manera acumulativa entre ejecuciones, evitando reconstrucciones integrales del entorno analítico.
- La integridad de los datos en cada ciclo se asegura mediante la gestión precisa de la marca de agua (*watermark*) y los filtros de deduplicación.
- La integración de transformacion.py asegura que los estándares de saneamiento y reglas de negocio permanezcan uniformes, mitigando el riesgo de discrepancias entre procesos.

La lógica general se encuentra centralizada en la función **procesar_incremental()**, cuya responsabilidad consiste en:

1. **Detección:** Identificación de los *deltas* (registros nuevos, modificados o eliminados) en el área de *staging* desde la última ejecución.
2. **Procesado:** Aplicación de la lógica de las Dimensiones que Cambian Lentamente (SCD) para actualizar los registros existentes o generar nuevas versiones históricas.
3. **Integración:** Sincronización y actualización de las dimensiones y tablas de hechos dentro del Data Warehouse (DWH).
4. **Validación:** Ejecución de controles de integridad para asegurar que el estado final del DWH cumpla con las reglas de negocio y consistencia esperadas.

Mecanismo de detección de cambios:

La estrategia consta de un sistema que utiliza una marca de agua temporal almacenada en una tabla de auditoría dentro de la misma base de datos de Staging. El campo **fecha_carga** (inyectado por carga_staging.py en cada registro al momento de la extracción) actúa como el indicador de novedad.

El mecanismo se basa en registrar el valor máximo del campo fecha_carga procesado exitosamente. De esta manera, la siguiente ejecución puede filtrar los registros de Staging cuya fecha_carga sea estrictamente superior a la **ultima_extraccion** registrada con estado "OK". Este diseño permite manejar de forma nativa las re-ejecuciones en caso de fallos, ya que si una carga queda en estado "RUNNING" o "ERROR", el sistema intenta procesar el mismo delta desde la última marca válida conocida.

Tabla de auditoría:

La marca de agua no se persiste en un archivo local sino directamente en la base de datos de Staging en la tabla **etl_auditoria_incremental**.

La marca de agua no se persiste en un archivo local sino directamente en la base de datos de Staging en la tabla **etl_auditoria_incremental**. Este enfoque garantiza la centralización de los metadatos de control, permitiendo que cualquier instancia del proceso ETL consulte el estado global de las cargas.

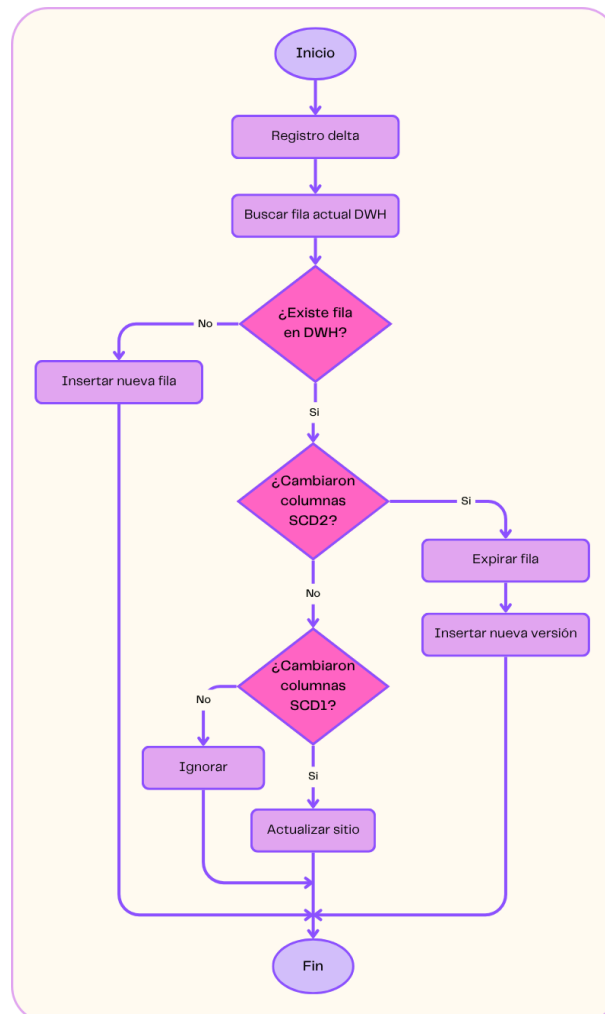
Columna	Tipo	Descripción
id	BIGINT AUTO_INCREMENT	Identificador de ejecución
inicio	DATETIME	Timestamp de inicio (UTC)

fin	DATETIME NULL	Timestamp de cierre (se escribe al finalizar)
ultima_extraccion	DATETIME NULL	Watermark de la ejecución anterior
nueva_extraccion	DATETIME NULL	Watermark que se guardará si la ejecución es exitosa
estado	VARCHAR(20) NOT NULL	RUNNING y OK o ERROR
registros_delta	INT DEFAULT 0	Cantidad total de registros procesados en el delta
mensaje_error	TEXT NULL	Detalle del error si el estado es ERROR

3. Manejo de SCD

El sistema implementa un SCD híbrido donde cada columna de una dimensión tiene asignada un tipo de cambio diferente según su relevancia analítica. El motor central es **aplicar_scd_generico()**, que es invocado tanto para **dim_estudiante** como para **dim_dictado** con listas de columnas diferenciadas.

El algoritmo ejecuta la siguiente secuencia:



Mecanismo SCD Tipo 2:

Cuando se detecta un cambio en una columna SCD-2, el proceso ejecuta dos operaciones en secuencia:

1. Expirar la fila actual (expirar_dimension):

```
UPDATE dim_estudiante -- o dim_dictado
SET   valid_to = CURDATE(),
      es_actual = FALSE
WHERE alumnoSKey = :sk_valor -- la surrogate key de la fila que expira
```

2. Insertar nueva fila:

El registro del delta se inserta como una fila nueva con:

- **valid_from = fecha de hoy** (establecida por la función constructora de la dimensión)
- **valid_to = NULL**
- **es_actual = TRUE**
- Una **nueva alumnoSKey** (o dictadoSKey) generada por AUTO_INCREMENT

Mecanismo SCD Tipo 1:

Cuando solo cambian columnas SCD-1 actualizar_dimension_scd1() ejecuta un UPDATE directo sobre la fila actual:

```
sql
UPDATE dim_estudiante
SET   genero = :genero,
      egreso_carrera = :egreso_carrera,
      ...
WHERE alumnoSKey = :sk_valor
```

Esta operación **no genera nueva fila ni expira la existente**: simplemente corrige el valor en el lugar. Los hechos históricos que apuntan a esa surrogate key verán el valor actualizado al consultar la dimensión, lo que es el comportamiento esperado para atributos SCD-1.

4. Descripción del flujo ETL

El flujo ETL desarrollado sigue las tres etapas clásicas: extracción, transformación y carga.

Proceso de extracción:

La extracción consiste en obtener los datos desde las tablas de staging utilizando consultas SQL simples ejecutadas mediante Pandas.

Durante esta etapa no se aplican transformaciones ni validaciones. El objetivo es recuperar el conjunto completo de registros disponibles para posteriormente ser procesados dentro de Python.

Proceso de transformación:

La etapa de transformación constituye el núcleo principal de la ETL. Aquí se realizan procesos de limpieza, normalización, validación, enriquecimiento y deduplicación de los datos. La lógica se encuentra centralizada en la clase DataCleaner, la cual encapsula funciones reutilizables de saneamiento de datos.

- **Limpieza de valores nulos semánticos:** Se identifican valores textuales que representan ausencia de información, tales como: "null", "none", "n/a", "sin dato", "s/d". Estos valores son convertidos explícitamente a None para garantizar consistencia durante el procesamiento posterior.
- **Normalización de formatos numéricos:** Los datos numéricos provenientes de staging pueden contener diferentes formatos regionales o inconsistencias. Por ello, la ETL implementa un proceso capaz de interpretar: separadores de miles, comas decimales, espacios, formatos mixtos. Esto permite convertir correctamente los valores hacia tipos enteros o flotantes independientemente del formato original.
- **Validación y limpieza de fechas:** La ETL soporta múltiples formatos de fecha mediante intentos secuenciales de parseo. Esto permite interpretar correctamente registros provenientes de fuentes heterogéneas. Entre los formatos soportados se incluyen: YYYY-MM-DD, DD/MM/YYYY, YYYYMMDD, DD-MM-YYYY, YYY.
- **Normalización de atributos categóricos:** Algunos atributos categóricos requieren una normalización. Por ejemplo, el género puede representarse como: "Masculino", "M", "Male", "Hombre". Todos estos valores son transformados a una representación estándar única. El mismo criterio se aplica para: estados de inscripción, resultados de examen, períodos académicos.
- **Validaciones de calidad:** La ETL implementa múltiples reglas de validación orientadas a asegurar integridad y coherencia: DNI válidos, notas entre 0 y 10, intentos mayores a cero, cupos positivos, claves obligatorias no nulas. Los registros que no cumplen las condiciones mínimas serán evaluados.
- **Eliminación de duplicados:** Otro aspecto importante corresponde a la deduplicación de registros. Para ello se utiliza drop_duplicates() sobre claves naturales específicas dependiendo de cada entidad. Esto evita inconsistencias y duplicidad analítica dentro del Data Warehouse.

Construcción del Modelo Dimensional:

Una vez finalizada la limpieza, la ETL construye las dimensiones y tablas de hechos.

- **Dimensión Tiempo:** La dimensión tiempo se genera dinámicamente a partir de todas las fechas presentes en: inscripciones, exámenes, evaluaciones. Para cada fecha se calculan atributos derivados como: día, mes, año, período académico.

La clave surrogate se construye utilizando el formato: AAAAMMDD.

- **Dimensión Estudiante:** La dimensión estudiante integra información personal y académica proveniente de distintas entidades relacionadas. Además de los datos básicos, se incorporan atributos derivados como: edad al ingreso, indicadores de abandono, datos del programa académico.
- **Dimensión Dictado:** La dimensión dictado constituye una dimensión altamente denormalizada que integra información de: cursos, docentes, departamentos y facultades. Este diseño busca reducir joins analíticos y facilitar consultas sobre rendimiento académico y evaluación docente.

Construcción de tablas de hechos:

Las tablas de hechos representan eventos medibles del dominio académico.

- **Fact Inscripción:** Representa cada inscripción de un estudiante a un dictado. Incluye métricas e indicadores como: estado de inscripción, abandono.
- **Fact Examen Estudiante:** Representa cada intento de examen realizado por un estudiante. Incluye: nota obtenida, número de intentos, condición de aprobación. La granularidad definida corresponde a: un intento de examen por estudiante y dictado.
- **Fact Evaluación Dictado:** Representa evaluaciones académicas realizadas sobre dictados. Incluye indicadores cuantitativos relacionados con: calidad del dictado, calidad de contenidos, valoración general.

Proceso de carga:

La carga hacia el Data Warehouse se realiza utilizando **to_sql()** de Pandas en combinación con SQLAlchemy. Para mejorar rendimiento y estabilidad, la inserción se ejecuta en lotes de 500 registros.

En caso de error durante la carga masiva, la ETL implementa un mecanismo de recuperación que intenta insertar los registros individualmente. Esto permite: aislar registros defectuosos, continuar la ejecución y registrar errores específicos en logs.

Paso a paso del flujo:

Este proceso completo se divide en dos scripts independientes que se ejecutan en secuencia: carga_staging.py y transformacion.py.

- **Carga_staging.py - Carga a staging:**

Se realizan los siguientes pasos:

1. **Diagnóstico pre-carga:** antes de procesar, el script verifica que cada archivo CSV fuente exista y que cada tabla de Staging correspondiente exista en MySQL. Si alguna condición falla, lo registra en el log.
2. **Limpieza previa de las tablas:** para cada tabla de Staging se ejecuta un TRUNCATE TABLE previo.
3. **Lectura de los sources:** se leen los archivos con pandas, agregando el sufijo como "str" para preservar exactamente el dato original.

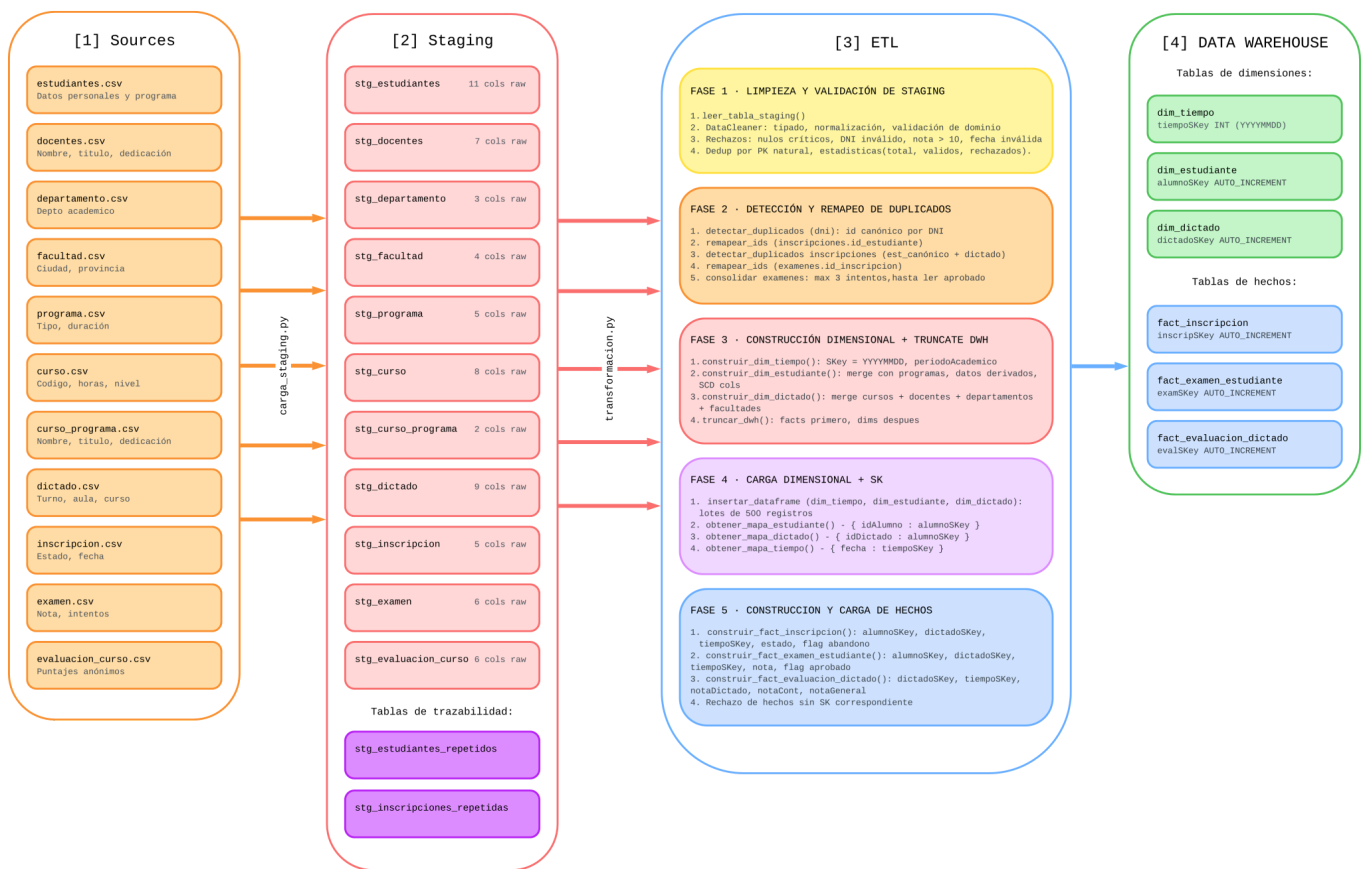
4. **Enriquecimiento especial de evaluacion_curso:** dado que la evaluación es anónima y el source original no contiene fecha, se infiere la fecha de evaluación con la fecha estimada del final del cuatrimestre al que pertenece (C1: 15/07, C2: 15/12). Si no puede determinarse, se usa la fecha actual como fallback.
5. **Renombrado de columnas:** todas las columnas adquieren el sufijo `_raw`.
6. **Inyección de metadatos:** se agregan `archivo_origen` y `fecha_carga`.
7. **Carga masiva.**

- **Transformacion.py - Transformación y carga al DW:**

La orquestación principal es la función `ejecutar_transformacion()`, que sigue estos 5 pasos:

1. **Limpieza y validación de Staging:** Se leen las 11 tablas de Staging y se aplica una función de transformación base a cada una. Cada función tipifica columnas, valida dominios y rechaza registros inválidos. Las estadísticas de rechazos y duplicados se registran por tabla.
2. **Detección y remapeo en cascada de duplicados:** Se detectan estudiantes con el mismo DNI conservando el primer id por DNI como el canónico. Luego se remapean las inscripciones y, en cascada, los exámenes. Finalmente se consolidan los intentos de examen eliminando según dos criterios: si ocurrieron dos o más intentos al mismo examen en un mismo día y, si el estudiante sigue rindiendo un mismo examen habiendo aprobado anteriormente con otro id.
3. **Construcción dimensional y truncado del DWH:** Se construyen en memoria `dim_tiempo` (desde todas las fechas de hechos), `dim_estudiante` (join con programas) y `dim_dictado` (join con cursos, docentes, departamentos, facultades). Después, se ejecuta el TRUNCATE del DWH en un orden seguro, primero hechos y luego dimensiones.
4. **Carga de dimensiones:** Las dimensiones se insertan en el DWH en lotes de 500 registros. Una vez insertadas, se consultan las surrogate keys generadas por MySQL para construir los mapas de resolución.
5. **Construcción y carga de tablas de hechos:** Se construyen `fact_inscripcion`, `fact_examen_estudiante` y `fact_evaluacion_dictado` resolviendo las surrogate keys mediante mapeo en memoria. Se insertan en lotes de 500 registros.

5. Diagrama del proceso



6. Principales transformaciones

1. Limpieza y normalización (clase DataCleaner): Todas las transformaciones de Staging se canalizan a través de la clase DataCleaner, que expone métodos estáticos reutilizables.

Método	Descripción	Reglas de dominio
<code>limpiar_string()</code>	Normaliza cadenas de texto	<code>strip()</code> + <code>title-case</code> . Valores nulos devuelven <code>None</code> .
<code>limpiar_numero(tipo)</code>	Convierte valores numéricos	Soporte para separadores de miles/decimales mixtos (punto y coma). Para enteros, elimina puntos. Para floats, preserva decimal. Valores "null/none/n/a" devuelven <code>None</code> .

limpiar_fecha()	Parseo multi-formato de fechas	8 formatos soportados: %Y-%m-%d, %d/%m/%Y, %Y%m%d, %d-%m-%Y, %m-%d-%Y, %Y, %d/%m/%y, %Y/%m/%d. Fallback con dayfirst=True.
limpiar_genero()	Normaliza género al dominio DWH	M: {M, MASCULINO, MALE, HOMBRE, 1}; F: {F, FEMENINO, FEMALE, MUJER, 2}. Cualquier otro valor devuelve None.
limpiar_dni()	Valida DNI argentino	Rango válido: 1.000.000 – 99.999.999. Devuelve un bool.
limpiar_nacionalidad()	Normaliza nacionalidad	Valores nulos devuelven "No especificado".

2. Detección y remapeo de duplicados: Cuando dos registros de estudiante (mismo DNI, distinto id) se fusionan, sus inscripciones y exámenes se solapan. La consolidación aplica las siguientes reglas sólo a los grupos afectados:

- Ordenar cronológicamente (por fecha, luego por id_examen).
- Si existe un examen aprobado entre todos sus intentos: truncar la serie en ese punto (todo lo posterior al aprobado se elimina).
- Si existen dos exámenes para el mismo dictado en la misma fecha exacta, se mantendrá el primer registro encontrado.
- Renumerar numero_intento desde 1 en orden.

3. Desnormalización para el modelo dimensional: La normalización del sistema se aplanar/desnormaliza en las dimensiones:

- dim_dictado concentra:
 - Datos del dictado (turno, aula, cupo, período).
 - Datos del curso (código, nombre, horas teóricas/prácticas/laboratorio, nivel).
 - Datos del docente (nombre, apellido, título, categoría, dedicación).
 - Datos del departamento y facultad (nombre, ciudad, provincia)
- dim_estudiante concentra:
 - Datos del estudiante (DNI, nombre, apellido, género, fecha de nacimiento, nacionalidad, año de ingreso).
 - Datos del programa (nombre, tipo, duración en años).
 - Campos derivados: edadIngreso (diferencia entre año de ingreso y año de nacimiento).

- Campos de control SCD Tipo 2: valid_from, valid_to, es_actual.

4. Generación de dim_tiempo: La dimensión tiempo se construye a partir del conjunto de fechas presentes en los hechos (inscripciones, exámenes, evaluaciones). La surrogate key sigue el patrón YYYYMMDD.

Cada registro de tiempo incluye: fecha, día, mes, año y período académico (C1-YYYY / C2-YYYY).

5. Resolución de Surrogate Keys para hechos: Las tablas de hechos no almacenan claves naturales. Tras insertar las dimensiones, se recuperan las SKs generadas por MySQL y se construyen mapas en memoria:

- a. Mapa_estudiante: idalumno (int) - alumnoSKey (int AUTO_INC).
- b. Mapa_dictado: idDictado (int) - dictadoSKey (int AUTO_INC).
- c. Mapa_tiempo: fecha (date) - tiempoSKey (int YYYYMMDD).

Los hechos se validan: se descartan registros donde alguna SK no pueda resolverse (clave huérfana).

6. Derivación de atributos:

- Aprobado en fact_examen_estudiante: se deriva del campo resultado, será True si el valor normalizado es "aprobado", False en caso contrario.
- Abandono en fact_inscripcion: se deriva del campo estado, será True si el valor normalizado pertenece al conjunto {"abandonada", "abandonado", "abandono", "baja"}.
- EdadIngreso: calculada como diferencia entre fecha de nacimiento y año de ingreso, ajustando por mes/día.
- EgresoCarrera / anioEgreso: inicializado en FALSE / NULL (para actualización futura).
- AbandonoCarrera / anioAbandono: inicializado en FALSE / NULL (para actualización futura).

7. Evidencias

Errores en los datos de la tabla Estudiantes:

Durante el análisis de los datos se encontraron diversos problemas en el source **Estudiantes.csv**. Entre los principales problemas tenemos:

- Duplicación en el campo **nombre_raw** y nulos en el campo **nacionalidad_raw**.

Problemas de la tabla de Inscripciones debido a la duplicación:

Debido a los registros duplicados de los estudiantes se expanden los errores al source **inscripciones.csv**. Esto debido a que el mismo estudiante con distintos **id_estudiante** puede inscribirse a distintos o el mismo dictado, acarreando el error anterior.

Ejemplo: Los id_estudiante 31817 y 31687 que pertenecen a Torres Laura, se registran a los mismos dictados dos veces, id_dictado 579 y 581:

[illegible]

Podemos observar un claro error debido a que un mismo estudiante se inscribe dos veces a los mismos dictados en fechas relativamente cercanas.

Problemas de la tabla de Exámenes debido a la duplicación:

En el source **exámenes.csv** ocurre un problema similar al acarrear el error de la tabla de inscripciones. Distintas **id inscripcion** pueden realizar el mismo intento del mismo examen.

Ejemplo: Podemos ver los id_inscripcion 245761 y 244816, que pertenecen a las inscripciones de Torres Laura al dictado 579 a través de los id_estudiante 31817 y 31687. Ambas inscripciones suman un total de 4 intentos de un mismo examen para el mismo dictado, cuando lo correcto sería permitir solo 3 intentos.

[illegible]

Corrección de errores a través de la tabla de equivalencias:

Se utilizan las tablas de equivalencia **stg_estudiantes_repetidos**, para mapear inscripciones desde **id_estudiante**, y **stg_inscripciones_repetidos**, para mapear exámenes desde **id_inscripcion**.

La tabla de estudiantes repetidos se construye comprobando los duplicados por **dni** y almacenando el primer valor de **id_estudiante** asociado a este como el **id_canonico**.

La tabla de inscripciones repetidas se construye comprobando solo los **id_inscripcion** que se mapearon previamente con el mismo **id_estudiante**, almacenando el primer valor de como el **id_canonico**.

Ejemplo: Podemos ver en **stg_estudiantes_repetidos** como se mapea la **id_estudiante** duplicada de Torres Laura a su id canónica **31687**.

```
1 • SELECT * FROM stg_universidad.stg_estudiantes_repetidos where id_repetido = 31817;
```

row_id	archivo_origen	fecha_carga	id_repetido	id_tomado
34	NULL	2026-05-13 12:35:47	31817	31687
NULL	NULL	NULL	NULL	NULL

El mismo caso se da en **stg_inscripciones_repetidos** mapendo a la **id_inscripción** canónica 244816.

```
1 • SELECT * FROM stg_universidad.stg_inscripciones_repetidas where id_repetido = 245761;
```

row_id	archivo_origen	fecha_carga	id_repetido	id_tomado
2161	NULL	2026-05-13 12:35:48	245761	244816
NULL	NULL	NULL	NULL	NULL

De esta forma al cargar el data warehouse solo se cargarán los valores canónicos y no los duplicados.

dim_estudiante: sin datos de estudiantes duplicados.

```
1 • SELECT * FROM dw_universidad.dim_estudiante where dni = 20236913;
```

alumnoKey	idalumno	dni	nombre	apellido	genero	fechaNac	nacionalidad	anioIngreso	edadIngreso	egresoCarrera	anioEgreso	abandonoCarrera	anioAbandono	nombrePrograma	tpoPrograma	duracionAni
1204	31687	20236913	Laura	Torres	F	2001-05-13	Argentina	2023	22	0	NULL	0	NULL	Ingeniería En Sistemas De Información	Grado	5
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

fact_inscripción: unión de todas las inscripciones pero sin los dictados duplicados.

```
1 • SELECT * FROM dw_universidad.fact_inscripcion where alumnoSKey = 1204;
```

InscripSKey	alumnoSKey	tiempoSKey	dictadoSKey	estado	abandono
7463	1204	20230224	574	Activa	0
7464	1204	20230227	575	Activa	0
7465	1204	20240203	577	Activa	0
7466	1204	20230219	579	Activa	0
7467	1204	20240221	581	Activa	0
7468	1204	20240322	584	Activa	0
7469	1204	20240305	585	Activa	0
7470	1204	20230303	587	Activa	0
7471	1204	20240214	589	Activa	0
7472	1204	20230323	591	Activa	0
7473	1204	20240213	592	Activa	0
7474	1204	20240305	597	Activa	0
7475	1204	20240211	600	Activa	0
7476	1204	20240222	601	Baja	1
7477	1204	20240211	608	Activa	0
7478	1204	20240301	609	Activa	0
7479	1204	20230323	636	Activa	0
7480	1204	20230205	637	Activa	0
7481	1204	20240305	639	Libre	0
7482	1204	20240214	642	Activa	0
NULL	NULL	NULL	NULL	NULL	NULL

fact_examen_estudiante: unión de todos los intentos de un mismo examen para el mismo dictado, truncando hasta el 1er intento aprobado y manteniendo solo los primeros registros de exámenes rendidos en la misma fecha exacta, reenumerando los intentos si es necesario.

8. Consideraciones de calidad de datos

El proceso implementa controles sobre: integridad, consistencia, unicidad, validez y normalización.

1. Validaciones aplicadas:

Dimensión	Mecanismo	Impacto si falla
Compleitud	Verificación de nulos en campos clave (id, nombre, DNI, fechas de hechos). Rechaza registros.	El registro se descarta. Se registra en log como "rechazado".
Validez de dominio	Géneros canónicos (M/F), DNI en rango 1M-99.99M, notas 0-10, intentos > 0, horas y cupos no negativos.	Anulación a NULL o descarte según criticidad del campo.
Consistencia de formato	8 formatos de fecha, 2 tipos numéricos con separadores regionales, title-case en strings.	Fallback progresivo hasta dayfirst=True. Si falla todo devuelve NULL y warning en log.
Unicidad	Deduplicación por clave primaria natural en cada tabla de staging. Detección de unicidad real por DNI para estudiantes.	Se conserva el primer registro (keep="first"). El resto se descarta y se registra.

Integridad referencial	Al construir hechos, sólo se aceptan registros con surrogate keys válidas en las tres dimensiones.	El hecho se descarta. Se reporta como "rechazado por claves no encontradas".
Trazabilidad	Todas las tablas STG guardan archivo_origen y fecha_carga. Duplicados se persisten en tablas de trazabilidad.	Permite auditar qué archivo generó cada registro y qué IDs fueron fusionados.

2. Manejo de datos faltantes:

- **Evaluaciones sin fecha:** se infiere la fecha desde el período del dictado. Si tampoco puede calcularse, se usa la fecha de ejecución del proceso como fallback documentado.
- **Estudiantes sin nacionalidad asociada:** se cargan en dim_estudiante con los atributos de nacionalidad como "sin especificar".
- **Estudiantes con fecha de nacimiento incompleta:** cuando en la fecha de nacimiento se recibe solo el año (YYYY) se infiere una fecha por defecto (YYYY/01/01).
- **Dictados con docente/departamento/facultad no encontrado:** se cargan en dim_dictado con esos atributos en NULL, registrando advertencia.
- **Hechos con surrogate keys no resolubles:** se descartan registrando el conteo de rechazados en el reporte.

9. Volumen y estructura de datos

1. Área de Staging - stg_universidad:

n°	Tabla	Columnas raw	Columnas técnicas	Clave primaria	Índices
1	stg_estudiante	11	3 (row_id, archivo_origen, fecha_carga)	row_id AUTO_INCREMENT	id_estudiante_raw
2	stg_docente	7	3	row_id AUTO_INCREMENT	id_docente_raw
3	stg_departamento	3	3	row_id AUTO_INCREMENT	id_departamento_raw
4	stg_facultad	4	3	row_id AUTO_INCREMENT	id_facultad_raw

5	stg_programa	5	3	row_id AUTO_INCREMENT	id_programa_raw
6	stg_curso	8	3	row_id AUTO_INCREMENT	id_curso_raw
7	stg_curso_prog rama	2	3	row_id AUTO_INCREMENT	id_curso_raw, id_programa_raw
8	stg_dictado	9	3	row_id AUTO_INCREMENT	id_dictado_raw
9	stg_inscripcion	5	3	row_id AUTO_INCREMENT	id_inscripcion_raw , id_estudiante_raw
10	stg_examen	6	3	row_id AUTO_INCREMENT	id_examen_raw, id_inscripcion_raw
11	stg_evaluacion _curso	6	3	row_id AUTO_INCREMENT	id_evaluacion_ra w, id_dictado_raw
stg_estudiantes_repetidos			Tabla auxiliar de trazabilidad de duplicados (id_repetido, id_tomado)		
stg_inscripciones_repetidas			Tabla auxiliar de trazabilidad de duplicados de inscripciones		
etl_auditoria_incremental			Tabla auxiliar de trazabilidad de procesos de cargas para calculo de delta_cambios		

2. Data Warehouse - dw_universidad:

Tabla	Tipo	Columnas	Surrogate Key	Restricciones notables
dim_tiempo	Dimensión	7	tiempoSKey INT (YYYYMMDD)	PK manual
dim_estudiante	Dimensión SCD-2	20	alumnoSKey AUTO_INC	CHECK vigencia, SCD campos

dim_dictado	Dimensión SCD-2	23	dictadoSKey AUTO_INC	CHECK vigencia, SCD campos
fact_inscripcion	Hecho	6	InscripSKey AUTO_INC	UNIQUE (alumno, dictado)
fact_examen_estudiante	Hecho	7	examAlumSK AUTO_INC	UNIQUE (alumno, dictado, intento)
fact_evaluacion_dictado	Hecho	6	evalDicSKey AUTO_INC	-

3. Granularidad de las tablas de hechos:

Hecho	Granularidad	Descripción
fact_inscripcion	1 fila por estudiante por dictado	Un estudiante se inscribe una vez en cada dictado.
fact_examen_estudiante	1 fila por estudiante por dictado por intento	Captura todos los intentos (o hasta el primero aprobado)
fact_evaluacion_dictado	1 fila por evaluación de dictado (anónima)	Registro de satisfacción del dictado.

10. Control de Carga a Data Warehouse

Para verificar la carga correcta de datos al Data Warehouse analizamos la tabla **stg_estudiantes** debido a que esta posee la problemática de duplicidad en los registros representando así un efecto en cascada en **stg_inscripciones** y **stg_exámenes**. Para esta comprobación de carga utilizamos queries de MySQL en las tablas staging las cuales cuentan con los datos fuentes comparándolas con los resultados del informe final de carga.

Control de Estudiantes:

Controlamos la salida de logs al realizar la transformación para comprobar la cantidad de estudiantes duplicados y los limpios cargados al final.

```
2026-05-16 19:56:59 - INFO - Detectar duplicados (estudiantes por DNI):
analizados=130000 | duplicados=2567 | limpios=127433
```

Después de realizar la comprobación de duplicidad en **stg_estudiantes** mediante MySQL podemos ver que los datos coinciden:

```
1 • SELECT SUM(cantidad - 1) AS total_solo_duplicados
2   FROM (
3     SELECT COUNT(*) AS cantidad
4     FROM stg_universidad.stg_estudiante
5     WHERE dni_raw IS NOT NULL
6     GROUP BY dni_raw
7     HAVING COUNT(DISTINCT id_estudiante_raw) > 1
8   ) AS agrupados;
```

Result Grid | | Filter Rows: | Export: | Wrap Cell Content:

	total_solo_duplicados
▶	2567

Así como los estudiantes cargados dentro de nuestra **dim_estudiante**:

```
1 • SELECT count(*) FROM dw_universidad.dim_estudiante where idAlumno;
```

Result Grid | | Filter Rows: | Export: | Wrap Cell Content:

	count(*)
▶	127433

Y la tabla **stg_estudiantes_repetidos**, la cual es crucial para el remapeo de inscripciones.

```
1 • SELECT count(*) FROM stg_universidad.stg_estudiantes_repetidos;
```

Result Grid | | Filter Rows: | Export: | Wrap Cell Content:

	count(*)
▶	2567

Control de Inscripciones:

La deduplicación de Estudiantes generó un conflicto en la carga debido a que muchos registros de Inscripciones quedaron huérfanas al eliminar los estudiantes a las que apuntaban. Observamos los logs de la ETL:

```
2026-05-11 23:54:45 - WARNING - Fact Inscripción por claves no encontradas: 19947 registros rechazados
```

Para controlar que todas las inscripciones fueron remapeadas correctamente comprobamos el reporte en logs:

```
2026-05-16 19:56:59 - INFO - Remapeo ids: columna='id_estudiante' |  
finalizado | remapeados=19947
```

Realizamos la comprobación mediante MySQL en **stg_inscripciones**:

```
1 • SELECT  
2     COUNT(i.id_inscripcion_raw) AS total_inscripciones_a_remapear  
3 FROM stg_universidad.stg_inscripcion i  
4 INNER JOIN stg_universidad.stg_estudiantes_repetidos r  
5     ON i.id_estudiante_raw = r.id_repetido;
```

Result Grid	Filter Rows:	Export:	Wrap Cell Content:
total_inscripciones_a_remapear			
19947			

Observamos que al fusionar estudiantes, un alumno queda inscrito dos veces al mismo dictado, eliminando dichas inscripciones duplicadas pero persistiendolas en la tabla **stg_inscripciones_repetidas** para posterior control de exámenes afectados.

```
2026-05-16 19:56:59 - INFO - stg_inscripciones_repetidas: 8819 mapeos  
persistidos
```

Comprobamos en MySQL que el número correcto de inscripciones duplicadas sean eliminadas:

```
20 -- Sumamos los excedentes (todo lo que sea mayor a 1 es un duplicado eliminado)  
21 SELECT  
22     SUM(cantidad - 1) AS inscripciones_repetidas_eliminadas  
23 FROM AgrupacionInscripciones  
24 WHERE cantidad > 1;
```

Result Grid	Filter Rows:	Export:	Wrap Cell Content:
inscripciones_repetidas_eliminadas			
8819			

Como control final vemos en los logs los reportes de carga de **fact_inscripción** para ver la cantidad final cargada.

```
2026-05-16 19:57:47 - INFO - fact_inscripcion: transformados=994594 |  
insertados=994594 | errores=0 | final=994594
```

Mediante MySQL contrastamos esta cantidad con la cantidad de **stg_inscripción** para comprobar que la cantidad de duplicados fue la cantidad eliminada finalmente.

```

1  SELECT
2      (SELECT COUNT(*) FROM stg_universidad.stg_inscripcion) AS cantidad_stg_inscripcion,
3      (SELECT COUNT(*) FROM dw_universidad.fact_inscripcion) AS cantidad_fact_inscripcion,
4      (SELECT COUNT(*) FROM stg_universidad.stg_inscripcion) - (SELECT COUNT(*) FROM dw_universidad.fact_inscripcion) AS diferencia_neta;

```

	cantidad_stg_inscripcion	cantidad_fact_inscripcion	diferencia_neta
▶	1003413	994594	8819

Control de Exámenes:

Como los exámenes tienen relación con las inscripciones de los alumnos, es necesario controlar su correcto remapeo debido a la eliminación de registros duplicados en **stg_inscripción**.

El reporte de logs de la ETL nos informa que se presentan 7862 remapeos de exámenes a sus id canónicos de inscripción.

```

2026-05-16 19:57:01 - INFO - Remapeo ids: columna='id_inscripcion' |
finalizado | remapeados=7862

```

Comprobamos en MySQL que el resultado sea el correcto:

```

3      SELECT
4          TRIM(id_repetido) AS id_inscripcion_mala,
5          TRIM(id_tomado) AS id_inscripcion_buena
6      FROM stg_universidad.stg_inscripciones_repetidas
7  )
8  -- Contamos los exámenes que están atados a esos IDs obsoletos
9  SELECT
10     COUNT(TRIM(e.id_examen_raw)) AS cantidad_examenes_remapeados
11  FROM stg_universidad.stg_examen e
12  INNER JOIN InscripcionesDuplicadas d
13      ON TRIM(e.id_inscripcion_raw) = d.id_inscripcion_mala;

```

	cantidad_examenes_remapeados
▶	7862

Con las inscripciones fusionadas, es posible que el alumno contenga en su historial de exámenes del mismo dictado instancias que antes estaban relacionadas a otros id.

Este conflicto no solo afecta a los exámenes remapeados sino también a aquellos relacionados a las inscripciones canónicas, debido a que hay que ordenarlos cronológicamente y reasignar su número de intento.

Afectando a 15.542 exámenes de 7.536 grupos que pertenecen al mismo “alumno-dictado”:

2026-05-16 19:57:01 - INFO - Consolidación exámenes: grupos impactados=7536 | exámenes impactados=15542

Mediante MySQL comprobamos que dicha cifra sea correcta:

```
1 WITH IDsImpactados AS (  
2   -- Paso 1: Extraemos y unificamos solo los IDs de inscripción que están en la limpieza  
3   SELECT TRIM(id_repetido) AS id_inscripcion FROM stg_universidad.stg_inscripciones_repetidas  
4   UNION  
5   SELECT TRIM(id_tomado) FROM stg_universidad.stg_inscripciones_repetidas  
6 ),  
7 IncripcionesAisladas AS (  
8   -- Paso 2: Buscamos SOLO esas inscripciones específicas y les resolvemos el estudiante  
9   SELECT  
10     TRIM(i.id_inscripcion_raw) AS id_inscripcion,  
11     COALESCE(TRIM(er.id_tomado), TRIM(i.id_estudiante_raw)) AS id_estudiante_canonico,  
12     TRIM(i.id_dictado_raw) AS id_dictado  
13   FROM stg_universidad.stg_inscripcion i  
14   -- El INNER JOIN acá actúa como un filtro temprano para no procesar toda la tabla  
15   INNER JOIN IDsImpactados ids
```

exámenes_impactados	grupos_impactados
15542	7536

Analizando los resultados observamos intentos de exámenes realizados en la misma fecha con resultados distintos. Ante esto tomamos como decisión truncar considerando el primer registro presente en dicha fecha como el correcto, eliminando el resto.

Esta consideración junto con la regla de negocio de truncar luego del primer aprobado nos generan una cantidad de 3290 exámenes eliminados:

026-05-16 19:57:09 - INFO - Consolidación exámenes: finalizado | afectados=15542 | eliminados=3290

Comprobamos dicho resultado en MySQL:

```
1 WITH IDsImpactados AS (  
2   -- 1. Aislamos las inscripciones  
3   SELECT TRIM(id_repetido) AS id_inscripcion FROM stg_universidad.stg_inscripciones_repetidas  
4   UNION  
5   SELECT TRIM(id_tomado) FROM stg_universidad.stg_inscripciones_repetidas  
6 ),  
7 IncripcionesAisladas AS (  
8   -- 2. Resolvemos los estudiantes  
9   SELECT  
10     TRIM(i.id_inscripcion_raw) AS id_inscripcion,  
11     COALESCE(TRIM(er.id_tomado), TRIM(i.id_estudiante_raw)) AS id_estudiante_canonico,  
12     TRIM(i.id_dictado_raw) AS id_dictado  
13   FROM stg_universidad.stg_inscripcion i  
14   INNER JOIN IDsImpactados ids
```

exámenes_eliminados
3290

Como medida de validación y para comprobar que los exámenes eliminados sean los correctos, comparamos la suma del total de notas de los exámenes analizados tanto en

staging y como en el Data Warehouse:

```
1 WITH IDsImpactados AS (  
2     SELECT TRIM(id_repetido) AS id_inscripcion FROM stg_universidad.stg_inscripciones_repetidas  
3     UNION  
4     SELECT TRIM(id_tomado) FROM stg_universidad.stg_inscripciones_repetidas  
5 ),  
6 InscripcionesAisladas AS (  
7     SELECT
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

suma_general_notas	cantidad_examenes_eliminados	suma_notas_eliminadas	resultado_final retenido
3955411.20	3290	13861.50	3941549.70

Comprobamos que dicho resultado está realmente presente en **fact_examen_estudiante**

```
1 SELECT SUM(nota) AS suma_total_notas  
2 FROM dw_universidad.fact_examen_estudiante;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

suma_total_notas
3941549.70